

Playwright Automation Training

AI-Powered Test Engineering | 5-Day Learning Path for QA Engineers

Field	Details
Program	Playwright AI-Powered Test Automation
Duration	5 Days — Hands-On Intensive
Target Audience	QA Engineers (Beginners to Playwright)
Language	TypeScript (JavaScript compatible)
AI Capabilities	AI Test Generation + Self-Healing Locators
Test Coverage	Web UI REST API Database
Prerequisite	Basic programming awareness; no Playwright experience needed

Program Objectives

- Expand Playwright knowledge to writing real, AI-enhanced test suites
- Cover Web UI, REST API, and Database testing in a unified Playwright framework
- Use AI (OpenAI/Claude) to auto-generate test cases from prompts and screenshots
- Implement self-healing locators so tests survive UI changes without manual fixes
- Understand where and how Playwright can be used for LLM Evals (feasibility assessment)
- Deliver a production-ready, CI-integrated test framework by Day 5

Note: There are given code samples, use them as reference and do not consider them as actual code for your project.

5- Day Program Overview

Day	Theme	Focus Areas	Deliverable
Day 1	PlaywrightFoundations	Setup, Core Concepts, First Tests, Selectors	Passing UI test suite
Day 2	API & DatabaseTesting	Playwright APIRequestContext, SQL assertions, Fixtures	API + DB test module
Day 3	AI TestGeneration	GPT/Claude prompt-to-test, screenshot-to-test, MCP	AI-generated test cases
Day 4	Excel & SwaggerPipeline	Excel parser, data-driven engine, Swagger-to-MD, typed tests	Zero-duplication QA pipeline
Day 5	Self-HealingLocators & CI	Locator strategies, AI fallback, resilience patterns, GitHub Actions, Evals Feasibility	Self-healing locator lib + CI

Playwright Foundations — From Zero to First Tests

Learning Goals

- Install and configure Playwright in a TypeScript project
- Understand Playwright's architecture: browsers, contexts, pages
- Write and run your first UI tests using best-practice selectors
- Understand test structure: describe/test/expect blocks
- Use Playwright Inspector and [Codegen](#) for rapid test authoring

1.1 Environment Setup

```
# Step 1: Install Node.js 18+ from https://nodejs.org
node --version # should print v18.x or higher

# Step 2: Create project and install Playwright
mkdir playwright-training && cd playwright-training
npm init -y
npm init playwright@latest
# Choose: TypeScript | tests/ folder | GitHub Actions | install browsers

# Step 3: Verify
npx playwright --version
npx playwright test --list
```

1.2 playwright.config.ts — Recommended Configuration

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  reporter: [['html'], ['list']],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
```

```

    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  ],
});

```

1.3 Your First UI Test

```

// tests/day1-basics.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Login Flow', () => {

  test('should login with valid credentials', async ({ page }) => {
    await page.goto('/login');
    await page.getByRole('textbox', { name: 'Username' }).fill('admin@example.com');
    await page.getByRole('textbox', { name: 'Password' }).fill('P@ssw0rd!');
    await page.getByRole('button', { name: 'Sign In' }).click();
    await expect(page).toHaveURL('/dashboard');
    await expect(page.getByRole('heading', { name: 'Welcome' })).toBeVisible();
  });

  test('should show error for invalid credentials', async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill('wrong@email.com');
    await page.getByLabel('Password').fill('wrongpass');
    await page.getByRole('button', { name: 'Sign In' }).click();
    await expect(page.getByText('Invalid credentials')).toBeVisible();
  });

});

```

1.4 Selector Priority Cheat Sheet

Selector Type	Example (Priority Order)
1. Role (ARIA) — Best	<code>page.getByRole('button', { name: 'Submit' })</code>
2. Label	<code>page.getByLabel('Email address')</code>
3. Placeholder	<code>page.getByPlaceholder('Enter email')</code>
4. Test ID	<code>page.getByTestId('login-btn')</code>
5. Text	<code>page.getByText('Sign In')</code>
6. CSS (last resort)	<code>page.locator('.btn-primary')</code>
AVOID: XPath	<code>// brittle — avoid in all new tests</code>

1.5 Running & Debugging Tests

```
npx playwright test                # run all tests headlessly
npx playwright test --headed        # visible browser
npx playwright test --ui            # interactive explorer
npx playwright codegen http://localhost:3000 # record from clicks
npx playwright test --debug         # step-through debugger
npx playwright show-report          # open HTML report
```

Day Deliverable

Passing UI test suite — minimum 5 tests (login success/failure, navigation x3) all green on Chromium + Firefox.

API & Database Testing — Full-Stack Assertions

Learning Goals

- Use Playwright's built-in APIRequestContext for REST API testing
- Validate response schemas with Zod
- Write database assertion tests using the pg (PostgreSQL) client
- Build shared fixtures for auth tokens and DB connections
- Combine UI + API + DB in a single test for full-stack validation

2.1 Install Dependencies

```
npm install zod                # schema validation
npm install pg @types/pg      # PostgreSQL client
npm install dotenv            # environment variables
```

2.2 API Testing with Playwright

```
// tests/day2-api.spec.ts
import { test, expect } from '@playwright/test';
import { z } from 'zod';

const UserSchema = z.object({
  id:      z.number(),
  email:   z.string().email(),
  role:    z.enum(['admin', 'user', 'viewer']),
});

test('GET /api/users returns paginated list', async ({ request }) => {
  const response = await request.get('/api/users?page=1&limit=10');
  expect(response.status()).toBe(200);
  const body = await response.json();
  expect(() => z.array(UserSchema).parse(body.data)).not.toThrow();
});

test('POST /api/users creates new user', async ({ request }) => {
  const response = await request.post('/api/users', {
    data: { email: 'test@example.com', username: 'testuser', role: 'user' }
  });
  expect(response.status()).toBe(201);
  const created = await response.json();
  expect(created.id).toBeDefined();
});
```

```
});
```

2.3 Auth Fixture — Reusable Across Tests

```
// tests/fixtures/db.fixture.ts
import { test as base, request } from '@playwright/test';

type AuthFixtures = { adminToken: string };

export const test = base.extend<AuthFixtures>({
  adminToken: async ({}, use) => {
    const ctx = await request.newContext({ baseURL: process.env.BASE_URL });
    const resp = await ctx.post('/api/auth/login', {
      data: { email: 'admin@example.com', password: process.env.ADMIN_PASS }
    });
    const { token } = await resp.json();
    await use(token);
    await ctx.dispose();
  },
});
```

2.4 Database Testing (PostgreSQL)

```
// tests/day2-database.spec.ts
import { test, expect } from '../fixtures/db.fixture';

test('new user persisted after API creation', async ({ request, db }) => {
  const resp = await request.post('/api/users', {
    data: { email: 'db-test@example.com', username: 'dbtest', role: 'user' }
  });
  const { id } = await resp.json();

  // Verify directly in database
  const result = await db.query('SELECT * FROM users WHERE id = $1', [id]);
  expect(result.rows.length).toBe(1);
  expect(result.rows[0].email).toBe('db-test@example.com');
});

test('password is never stored in plain text', async ({ db }) => {
  const result = await db.query(
    "SELECT password_hash FROM users WHERE email = 'admin@example.com'"
  );
  // bcrypt hashes always start with $2b$
  expect(result.rows[0].password_hash).toMatch(/^$2b$/);
});
```

Day Deliverable

API + DB test module — 8 API tests (GET/POST/PUT/DELETE with schema validation) + 4 DB integrity tests. All DB tests use ROLLBACK transactions — zero data pollution.

AI-Assisted Test Generation — Prompts, Screenshots & MCP

Learning Goals

- Use AI (Claude/GPT) to generate Playwright tests from plain-English descriptions
- Convert screenshots of UI into working test cases
- Generate tests from Swagger/OpenAPI 3.0 specifications
- Use Model Context Protocol (MCP) for IDE-integrated AI test generation
- Author SKILL.md and AGENT.md files to give AI tools persistent, reusable context
- Review, refine, and maintain AI-generated tests

3.1 Master Prompt Template

You are an expert QA automation engineer specialising in Playwright (TypeScript).

Generate a complete, runnable Playwright test file for the following:

APPLICATION: [App name and brief description]

BASE URL: [https://your-app.com]

FEATURE: [Feature name, e.g. 'User Registration']

TEST CASES TO COVER:

1. [Happy path description]
2. [Error / edge case description]
3. [Boundary condition]

CONSTRAINTS:

- Use getByRole() and getByLabel() selectors — avoid CSS/XPath
- Add data-testid where needed and note them
- Include beforeEach for shared setup
- TypeScript strict-mode compatible
- Add /* AI-GENERATED — Review required */ header comment

3.2 Example AI-Generated Test

```
/* AI-GENERATED — Review required */  
// Generated: Claude claude-sonnet-4-20250514 | Reviewed by: [Name] | Date: [YYYY-MM-DD]  
  
import { test, expect } from '@playwright/test';
```

```

test.describe('User Authentication', () => {

  test.beforeEach(async ({ page }) => {
    await page.goto('/auth/login');
    await expect(page.getByRole('heading', { name: 'Sign In' })).toBeVisible();
  });

  test('should login successfully with valid credentials', async ({ page }) => {
    await page.getByLabel('Email').fill('admin@example.com');
    await page.getByLabel('Password').fill('SecureP@ss123');
    await page.getByRole('button', { name: 'Sign In' }).click();
    await expect(page).toHaveURL('/dashboard');
    await expect(page.getByTestId('user-avatar')).toBeVisible();
  });

  test('should lock account after 5 failed attempts', async ({ page }) => {
    for (let i = 0; i < 5; i++) {
      await page.getByLabel('Email').fill('victim@example.com');
      await page.getByLabel('Password').fill(`wrong-${i}`);
      await page.getByRole('button', { name: 'Sign In' }).click();
      await page.getByLabel('Email').clear();
    }
    await expect(page.getByText('Account temporarily locked')).toBeVisible();
  });

});

```

3.3 Screenshot-to-Test Prompt

I am attaching a screenshot of [page name] from our application.

Please analyse the UI elements visible and generate:

1. A complete Playwright test covering the main user flow
2. Three edge-case tests for form validation
3. Recommended data-testid attributes to add to the HTML

Assume the base URL is <https://app.example.com>.

Use Playwright best-practice selectors and add comments.

3.4 MCP Integration for VS Code

```

// .mcp/playwright-mcp.json
{
  "mcpServers": {
    "playwright": {
      "command": "npx",

```

```

    "args": ["@playwright/mcp@latest"],
    "env": { "PLAYWRIGHT_HEADLESS": "false" }
  }
}

// With MCP active, prompt Claude in VS Code:
// "Generate a Playwright test for the checkout flow on localhost:3000"
// Claude opens the browser, explores the page, and writes tests automatically.

```

```

// tests/day3-ai-generated/login-ai.spec.ts (example of saved AI output)
// utils/ai-test-generator.ts             - helper for AI API calls
// utils/screenshot-to-test.ts           - screenshot → prompt → test

```

3.5 Skills & Agents — Teaching AI Your Team's Rules

Instead of pasting the same constraints into every prompt, you can write a **SKILL.md** or **AGENT.md** file that an AI tool reads once and follows permanently. Think of it as a standing instruction manual your AI colleague always has open before it writes a single line of test code.

File	What it is	When AI reads it
SKILL.md	Teaches AI one specific task — e.g. how YOUR team writes Playwright tests	You include it in your prompt or IDE context window
AGENT.md	Top-level rules for an autonomous agent session (naming, patterns, output format)	Agent reads it at the start of every session automatically
.cursorrules / .clinerules	Cursor / Cline IDE equivalents — same idea, tool-specific filename	The IDE injects it silently before every AI interaction
system-prompt.md	Claude Projects / GPT Custom Instructions equivalent	Loaded once per project workspace by the AI platform

3.5.1 Writing Your Team's SKILL.md

A SKILL.md is a plain Markdown file. It has three parts: a description that tells the AI what the skill is for, a rules section with your non-negotiable conventions, and a worked example the AI can imitate.

```

# skills/playwright-test-writer/SKILL.md

## What this skill does
Generates production-ready Playwright TypeScript test files for our application.
Read this file in full before writing any test code.

## Non-negotiable rules
1. ALWAYS use getByRole() or getByLabel() as the first-choice selector.
   Never use CSS classes or XPath unless all other options are exhausted.

```

```

2. ALWAYS wrap each test file in test.describe() with a meaningful name.
3. ALWAYS add the review header at the top of every generated file:
    /* AI-GENERATED - Review required | Engineer: [name] | Date: [YYYY-MM-DD] */
4. NEVER hard-code URLs - use process.env.BASE_URL or the baseUrl config.
5. ALWAYS follow AAA structure: Arrange → Act → Assert with blank lines between.
6. ALWAYS import from '@playwright/test', never from 'playwright' directly.
7. Test names MUST follow: 'should [action] when [condition]'.
8. DB tests MUST use the db.fixture.ts and rely on ROLLBACK - never commit.

## Selector priority order (follow strictly)
1. getByRole()           - best, tied to accessibility
2. getLabel()            - form fields
3. getByPlaceholder()    - inputs without a label
4. getByTestId()         - use data-testid attributes
5. getByText()           - for visible text content
6. locator('css')        - last resort only

## Output format
Return ONLY the TypeScript file content.
No explanations, no markdown fences, no preamble - just the file.

## Worked example
INPUT: 'Test that an admin can delete a user and the user disappears from the list'

OUTPUT:
/* AI-GENERATED - Review required | Engineer: | Date: */
import { test, expect } from '@playwright/test';
import { test as dbTest } from '../fixtures/db.fixture';

test.describe('Admin User Management', () => {
  test('should remove user from list when admin deletes them', async ({ page }) => {
    // Arrange
    await page.goto('/admin/users');
    const targetRow = page.getByRole('row', { name: 'jane@example.com' });
    await expect(targetRow).toBeVisible();

    // Act
    await targetRow.getByRole('button', { name: 'Delete' }).click();
    await page.getByRole('button', { name: 'Confirm' }).click();

    // Assert
    await expect(targetRow).not.toBeVisible();
  });
});

```

3.5.2 Example AGENT.md for Autonomous Test Generation

An AGENT.md goes one level higher — it governs an entire agent session, not just one file. Use it when you want Claude Code, Cursor, or Cline to autonomously explore a feature and generate multiple tests without you guiding each step.

```
# AGENT.md (place in project root — read automatically by Claude Code / Cursor)

## Role
You are a senior QA automation engineer on this project.
Your job is to write Playwright TypeScript tests that follow our team conventions.
Always read skills/playwright-test-writer/SKILL.md before generating any test.

## Project context
- Framework : Playwright + TypeScript
- Base URL   : process.env.BASE_URL (set in .env)
- Database   : PostgreSQL — always use tests/fixtures/db.fixture.ts
- Auth       : JWT — obtain token via POST /api/auth/login, store in fixture
- Page Objects: all page interactions go through pages/*.ts
- Test data  : never hard-code — use test-data/test-cases.xlsx or faker

## What you may do autonomously
- Read any file in the repository
- Create new .spec.ts files inside tests/
- Create new page objects inside pages/
- Run 'npx playwright test <file>' to verify tests pass

## What you must NOT do without asking
- Modify playwright.config.ts
- Modify any existing passing test
- Commit or push to git
- Install new npm packages

## Output checklist (verify before finishing)
[ ] Review header comment present in every generated file
[ ] All selectors follow the priority order in SKILL.md
[ ] No hard-coded URLs or credentials
[ ] All DB tests use ROLLBACK fixture
[ ] npx playwright test <new-file> exits with code 0
```

3.5.3 Tool-Specific Integration

AI Tool	Skill/Agent file location	How to activate
Claude Code (CLI)	AGENT.md in project root	Reads automatically on every session start
Cursor IDE	.cursorsrules in project root	Injected before every AI chat and Cmd+K action

Cline (VS Code ext.)	.clinerules in project root	Injected before every Cline task
Claude Projects	Project Instructions field in Claude.ai	Paste SKILL.md content into the instructions box
GPT Custom GPT	System prompt field in GPT builder	Paste SKILL.md content as system instructions
Copilot (GitHub)	.github/copilot-instructions.md	Auto-read by Copilot Chat in VS Code
Continue.dev	.continue/config.json → systemMessage field	Add SKILL.md content to the systemMessage string

3.5.4 Using the Skill Inside a Prompt

When your AI tool does not auto-read skill files, include the file content explicitly using one of these patterns:

```
// Pattern A – Paste skill inline (any AI tool)
/*
  <skill>
  [paste full contents of skills/playwright-test-writer/SKILL.md here]
  </skill>

  Using the skill above, generate a Playwright test for:
  'User can reset password via the Forgot Password link'
*/

// Pattern B – Reference by path (Claude Code / Cursor with file access)
/*
  Read skills/playwright-test-writer/SKILL.md first.
  Then generate a test for the shopping cart checkout flow.
*/

// Pattern C – System prompt injection (Claude Projects / GPT)
// In the Project Instructions / System Prompt box, add:
// 'Before writing any Playwright test, follow the rules in the
// attached SKILL.md exactly.'
// Then attach SKILL.md as a project file.
```

3.5.5 Skill File Folder Structure

```
playwright-training/
├─ skills/
│   └─ playwright-test-writer/
│       └─ SKILL.md          # Core Playwright conventions for AI
│   └─ api-test-writer/
│       └─ SKILL.md          # API testing rules (Zod, fixtures, status codes)
│   └─ db-test-writer/
│       └─ SKILL.md          # DB rules (always ROLLBACK, never SELECT *)
```

```
└─ excel-driven/  
  └─ SKILL.md          # How to parse Excel and map to test steps
```

Root-level agent files (auto-read by tools that support them)

AGENT.md # Claude Code – full project context

.cursorrules # Cursor IDE

.clinerules # Cline VS Code extension

.github/copilot-instructions.md # GitHub Copilot

Team Exercise — Write Your First SKILL.md

- Pick one convention your team argued about in the last sprint (e.g. 'should we use data-testid or role selectors?').
- Write a SKILL.md that settles it with a clear rule and a worked example.
- Test it: paste the skill into Claude or Cursor and ask it to generate a test. Did it follow the rule?
- Commit the skill file to the repo under skills/. It's now part of your team's shared AI configuration.

Day Deliverable

10+ AI-generated tests across UI, API, and DB + a SKILL.md for Playwright test writing + an AGENT.md for autonomous generation. Each generated file includes the review header. A prompt-library.md documents the team's top 5 prompts.

Excel & Swagger Pipeline — Zero-Duplication Data-Driven Tests

Learning Goals

- Parse Excel test case files and drive Playwright from them
- Define and enforce the standard Excel test case template
- Generate typed TypeScript interfaces from OpenAPI 3.0 specs
- Auto-generate Markdown API docs from a Swagger URL
- Build a zero-duplication pipeline: one source of truth per test

4.1 Excel Test Case Template — Column Schema

Column	Type & Description
TestID	String — Unique ID, e.g. TC-001
Module	String — Feature area, e.g. Login, Checkout
TestName	String — Human-readable description
Priority	Enum — High Medium Low
InputData	JSON String — e.g. {"email":"a@b.com","expectedUrl":"/dashboard"}
Steps	String — Pipe-separated: 'Navigate Fill form Submit'
ExpectedResult	String — Expected outcome description
TestType	Enum — UI API DB E2E
Enabled	Boolean — TRUE / FALSE (FALSE rows are skipped)

4.2 Excel Parser — utils/excel-reader.ts

```
import * as xlsx from 'xlsx';
import { z } from 'zod';

const TestCaseSchema = z.object({
  TestID: z.string(),
  Module: z.string(),
  TestName: z.string(),
  Priority: z.enum(['High', 'Medium', 'Low']),
  InputData: z.string().transform(s => { try { return JSON.parse(s); } catch { return {}; } }),
  Steps: z.string().transform(s => s.split('|').map(t => t.trim())),
  ExpectedResult: z.string(),
  TestType: z.enum(['UI', 'API', 'DB', 'E2E']),
  Enabled: z.union([z.boolean(), z.string()])
    .transform(v => String(v).toUpperCase() === 'TRUE'),
```



```

});
export type TestCase = z.infer<typeof TestCaseSchema>;

export function parseExcelTestCases(filePath: string): TestCase[] {
  const wb = xlsx.readFile(filePath);
  const ws = wb.Sheets['TestCases'];
  return xlsx.utils.sheet_to_json(ws, { defval: '' })
    .map(row => TestCaseSchema.safeParse(row))
    .filter(r => r.success && r.data.Enabled)
    .map(r => (r as any).data);
}

```

4.3 Data-Driven Test Engine — tests/day6-excel-driven.spec.ts

```

import { test, expect } from '@playwright/test';
import { parseExcelTestCases } from '../utils/excel-reader';

const cases = parseExcelTestCases('./test-data/test-cases.xlsx');
const uiCases = cases.filter(tc => tc.TestType === 'UI');

test.describe('Excel-Driven UI Tests', () => {
  for (const tc of uiCases) {
    test(`[${tc.TestID}] ${tc.TestName}`, async ({ page }) => {
      await page.goto(tc.InputData.url || '/');
      if (tc.InputData.username)
        await page.getByLabel('Username').fill(tc.InputData.username);
      if (tc.InputData.submitButton)
        await page.getByRole('button', { name: tc.InputData.submitButton }).click();
      if (tc.InputData.expectedUrl)
        await expect(page).toHaveURL(new RegExp(tc.InputData.expectedUrl));
      if (tc.InputData.expectedText)
        await expect(page.getByText(tc.InputData.expectedText)).toBeVisible();
    });
  }
});

```

4.4 Swagger URL → Markdown Docs — scripts/swagger-to-md.ts

(Suggestion: Feel free create your own script to create swaager-context.md file)

```

import SwaggerParser from '@apidevtools/swagger-parser';
import fs from 'fs';

async function swaggerToMd(specUrl: string): Promise<void> {
  const api = await SwaggerParser.parse(specUrl) as any;
  let md = `# API Reference — ${api.info.title}\n\n`;
}

```

```

for (const [path, methods] of Object.entries(api.paths || {})) {
  for (const [method, op] of Object.entries(methods as any)) {
    if (['get', 'post', 'put', 'patch', 'delete'].includes(method)) {
      md += `## ${method.toUpperCase()} ${path}\n`;
      md += `**Summary:** ${(op as any).summary}\n\n`;
    }
  }
}

fs.writeFileSync('./docs/api-reference.md', md);

console.log('✔ docs/api-reference.md generated');
}

swaggerToMd(process.env.SWAGGER_URL!);

```

4.5 Auto-Generate TypeScript Types from OpenAPI

```

# Install openapi-typescript
npm install -D openapi-typescript

# Generate src/api-client/Api.ts from Swagger URL
npx openapi-typescript $SWAGGER_URL --output ./src/api-client/Api.ts

# Add to package.json scripts:
# "gen:types": "openapi-typescript $SWAGGER_URL -o ./src/api-client/Api.ts"
# "gen:docs": "ts-node scripts/swagger-to-md.ts"

```

```

// tests/day6-swagger-api.spec.ts - typed API tests using generated Api.ts
import { test, expect } from '@playwright/test';
import type { components } from '../src/api-client/Api';

type User = components['schemas']['User'];

test('GET /api/users returns valid User array', async ({ request }) => {
  const resp = await request.get('/api/users');
  const body: { data: User[] } = await resp.json();
  expect(body.data[0].role).toMatch(/^(admin|user|viewer)$/);
});

```

Day Deliverable

Zero-duplication QA pipeline — Excel parser loading 20+ test cases with Zod validation + TypeScript types auto-generated from Swagger spec. Single 'npm run test:all' runs everything.

Self-Healing Locators & CI/CD — Production-Grade Framework

Learning Goals

- Understand why locators break and how to prevent it
- Build a self-healing locator utility with fallback chains
- Implement Page Object Model (POM) for maintainable tests
- Configure GitHub Actions for automated test runs on every PR
- Publish HTML reports as GitHub artifact — zero broken main branch

5.1 Self-Healing Locator Library — utils/self-healing-locator.ts

```
import { Page, Locator } from '@playwright/test';

interface Strategy { name: string; selector: string; priority: number; }

export class SelfHealingLocator {
  private strategies: Strategy[];

  constructor(private page: Page, strategies: Strategy[]) {
    this.strategies = strategies.sort((a, b) => a.priority - b.priority);
  }

  async find(timeout = 5000): Promise<Locator> {
    for (const s of this.strategies) {
      try {
        const loc = this.page.locator(s.selector);
        await loc.waitFor({ state: 'visible', timeout });
        if (s.priority > 1) console.warn(`[SelfHealing] Fell back to '${s.name}'`);
        return loc;
      } catch { /* try next */ }
    }
    throw new Error(`[SelfHealing] All strategies failed for element`);
  }
}

// Usage:
// const btn = new SelfHealingLocator(page, [
//   { name: 'testId', selector: '[data-testid="submit"]', priority: 1 },
//   { name: 'role', selector: 'button[name="Submit"]', priority: 2 },
//   { name: 'text', selector: 'text=Submit', priority: 3 },
```

```
// });  
// await (await btn.find()).click();
```

5.2 Page Object Model — pages/LoginPage.ts

```
import { Page, expect } from '@playwright/test';  
import { SelfHealingLocator } from '../utils/self-healing-locator';  
  
export class LoginPage {  
  private emailInput = new SelfHealingLocator(this.page, [  
    { name: 'label', selector: 'input[aria-label="Email"]', priority: 1 },  
    { name: 'testId', selector: '[data-testid="email-input"]', priority: 2 },  
    { name: 'name', selector: 'input[name="email"]', priority: 3 },  
  ]);  
  
  constructor(private page: Page) {}  
  
  async navigate() { await this.page.goto('/login'); }  
  
  async login(email: string, password: string) {  
    await (await this.emailInput.find()).fill(email);  
    await this.page.getByLabel('Password').fill(password);  
    await this.page.getByRole('button', { name: 'Sign In' }).click();  
  }  
  
  async expectError(message: string) {  
    await expect(this.page.getByRole('alert')).toContainText(message);  
  }  
}
```

5.3 GitHub Actions CI Pipeline

```
# .github/workflows/playwright.yml  
name: Playwright Tests  
on:  
  push: { branches: [main, develop] }  
  pull_request: { branches: [main] }  
  schedule: [{ cron: '0 6 * * *' }] # daily at 06:00 UTC  
  
jobs:  
  test:  
    runs-on: ubuntu-latest  
    services:  
      postgres:  
        image: postgres:15  
        env: { POSTGRES_DB: testdb, POSTGRES_USER: postgres, POSTGRES_PASSWORD:
```

```

password }
  ports: ['5432:5432']
  options: --health-cmd pg_isready --health-interval 10s
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: '20', cache: 'npm' }
  - run: npm ci
  - run: npx playwright install --with-deps
  - run: npx playwright test
    env: { BASE_URL: ${ secrets.BASE_URL }, DB_HOST: localhost }
  - uses: actions/upload-artifact@v4
    if: always()
    with:
      name: playwright-report-${ github.run_number }
      path: playwright-report/
      retention-days: 30

```

5.4 npm Scripts — Complete Pipeline

```

{ "scripts": {
  "gen:types": "openapi-typescript $SWAGGER_URL -o ./src/api-client/Api.ts",
  "gen:docs": "ts-node scripts/swagger-to-md.ts",
  "test:ui": "playwright test tests/day1-basics.spec.ts",
  "test:api": "playwright test tests/day2-api.spec.ts",
  "test:db": "playwright test tests/day2-database.spec.ts",
  "test:ai": "playwright test tests/day3-ai-generated/",
  "test:excel": "playwright test tests/day6-excel-driven.spec.ts",
  "test:all": "npm run gen:types && playwright test",
  "test:smoke": "playwright test --grep @smoke",
  "report": "playwright show-report"
} }

```

5.5 Perform Evals feasibility check

- Playwright by identifying key user flows and validating whether they can be reliably automated.
- UI and interaction testing
- Workflow validation (query → retrieval → response)
- Basic assertions on outputs

Day Deliverable

Self-healing locator library + all page classes refactored to POM + GitHub Actions CI green on main. HTML report artifact published on every run. Zero test failures enforced via branch protection.

Sample Final Repository Structure

After completing all 5 days, your repository should match this structure exactly. Each file maps to the day it was created.

```
playwright-training/
├── test-data/
│   ├── test-cases.xlsx           # Excel test cases (QA-maintained)
│   └── test-cases.example.xlsx   # Template with column headers
├── docs/
│   └── api-reference.md          # Auto-generated from Swagger URL
├── src/
│   └── api-client/
│       └── Api.ts                # Auto-generated by openapi-typescript
├── tests/
│   ├── day1-basics.spec.ts       # Day 1 - UI foundations
│   ├── day2-api.spec.ts          # Day 2 - API testing
│   ├── day2-database.spec.ts     # Day 2 - DB assertions
│   ├── day3-ai-generated/        # Day 3 - AI-generated tests
│   │   └── login-ai.spec.ts
│   ├── day4-self-healing.spec.ts # Day 5 - Self-healing locators
│   ├── day5-pom.spec.ts          # Day 5 - Full POM tests
│   ├── day5-ai-eval.spec.ts      # Day 5 - LLM Eval feasibility
│   ├── day6-excel-driven.spec.ts # Day 4 - Data-driven from Excel
│   ├── day6-swagger-api.spec.ts  # Day 4 - Tests from Swagger spec
│   └── fixtures/
│       └── db.fixture.ts         # Shared DB + auth fixtures
├── pages/
│   └── LoginPage.ts              # Page Object Model
├── utils/
│   ├── excel-reader.ts           # ExcelJS parser + Zod validation
│   ├── ai-test-generator.ts      # AI API call helper
│   ├── screenshot-to-test.ts     # Screenshot → prompt → test
│   └── self-healing-locator.ts   # Multi-strategy fallback locator
├── scripts/
│   ├── swagger-to-md.ts          # Swagger URL → docs/api-reference.md
│   └── generate-tests.ts         # AI batch test generation
├──
├── skills/                       # — AI Skill files (Day 3) —————
│   ├── playwright-test-writer/
│   │   └── SKILL.md              # Core Playwright conventions for AI
│   ├── api-test-writer/
│   │   └── SKILL.md              # API testing rules (Zod, fixtures)
│   ├── db-test-writer/
│   │   └── SKILL.md              # DB rules (always ROLLBACK)
│   └── excel-driven/
│       └── SKILL.md              # Excel-to-test mapping rules
├──
```

```
├─ .github/
│   └─ workflows/
│       └─ playwright.yml          # CI: test + upload HTML report
│       └─ copilot-instructions.md # GitHub Copilot skill instructions
├─ AGENT.md                       # Claude Code – full project context
├─ .cursorsrules                  # Cursor IDE – auto-injected rules
├─ .clinerules                    # Cline VS Code ext – auto-injected rules
├─ playwright.config.ts
├─ .env                           # BASE_URL, DB_HOST, SWAGGER_URL, etc.
├─ package.json
└─ tsconfig.json
```

Quick Reference — Key Commands

Command	Purpose
<code>npm run test:all</code>	Generate types + run full suite
<code>npm run gen:types</code>	Regenerate API types from Swagger
<code>npm run gen:docs</code>	Regenerate Markdown API docs
<code>npx playwright test --ui</code>	Interactive test explorer
<code>npx playwright codegen <url></code>	Record test from browser actions
<code>npx playwright show-report</code>	Open last HTML report
<code>npx playwright test --debug</code>	Step-through debugger
<code>npx playwright test --grep @smoke</code>	Smoke tests only
